ExploratoryGenius

```
# Load in jsonlite package
library(jsonlite)
library(dplyr)
```

```
Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```
library(igraph)
```

```
Attaching package: 'igraph'

The following objects are masked from 'package:dplyr':

    as_data_frame, groups, union

The following objects are masked from 'package:stats':

    decompose, spectrum
```

```
The following object is masked from 'package:base':

    union
```

```
#Assign Node Data and get a snapshot

#datausers_nominal <- stream_in(file("/Users/alexgould/Desktop/STA585GeniusProject/genius-exp
#head(datausers)
#data_annotate <- stream_in(file(("//Users/alexgould/Desktop/STA585GeniusProject/genius-exper
#head(data_annotate)
#songinfo <-  stream_in(file(("/Users/alexgould/Desktop/STA585GeniusProject/genius-expertise/
#head(songinfo)
# save(datausers_nominal, data_annotate, songinfo, file = "/Users/alexgould/Desktop/STA585Ge

# Load in Node data
load("/Users/alexgould/Desktop/STA585GeniusProject/initialfiles.Rdata")

# Assign Edge Data
edges_nominal <- read.csv("/Users/alexgould/Desktop/STA585GeniusProject/genius-expertise/foll

## ---genre first, computed from ALL annotations of every annotator ----

top_tag <- function(x) {
  x <- unlist(x)
  if (length(x) == 0) NA_character_ else x[1]
}
song_tags <- data.frame(
  key = tolower(songinfo$url_name),
  tag = vapply(songinfo$tags, top_tag, character(1)),
  stringsAsFactors = FALSE
)
song_tags <- song_tags[!duplicated(song_tags$key), ]

annot <- data_annotate[!is.na(data_annotate$user), ]
annot$tag <- song_tags$tag[match(tolower(annot$song), song_tags$key)]
mean(is.na(annot$tag))                # share of annotations with no genre
```

```
[1] 0.001465108
```

```
mode_tag <- function(x) {
  x <- x[!is.na(x)]
  if (length(x) == 0) return(NA_character_)
```

```
  tb  <- table(x)
  top <- names(tb)[tb == max(tb)]
  if (length(top) == 1) top else sample(top, 1)
}
set.seed(585)
user_tag  <- tapply(annot$tag, annot$user, mode_tag)
user_tags <- data.frame(url_name = names(user_tag),
                        tag = as.character(user_tag),
                        stringsAsFactors = FALSE)
user_tags <- user_tags[!is.na(user_tags$tag), ]   # genre is required to be in the study

## ---- CHANGED: study population ----
# Annotator with a genre + a profile + observed in the crawl (either column).
# (Old code required BOTH columns, which excluded users who only follow or are only followed
in_network <- union(edges_nominal$follower, edges_nominal$following)

study_users <- Reduce(intersect, list(user_tags$url_name,
                                      datausers_nominal$url_name,
                                      in_network))
length(study_users)
```

```
[1] 9121
```

```
# Edges among study users, no self-loops, no duplicate rows
edges_study <- edges_nominal[edges_nominal$follower  %in% study_users &
                             edges_nominal$following %in% study_users &
                             edges_nominal$follower != edges_nominal$following,
                             c("follower", "following")]
edges_study <- unique(edges_study)

# Nodes: ALL study users, isolates included
nodes_one   <- datausers_nominal[!duplicated(datausers_nominal$url_name) &
                                 datausers_nominal$url_name %in% study_users, ]
nodes_study <- merge(nodes_one, user_tags, by = "url_name")

stopifnot(!anyDuplicated(nodes_study$url_name),
          nrow(nodes_study) == length(study_users),
          all(unlist(edges_study) %in% nodes_study$url_name))

sort(table(nodes_study$tag), decreasing = TRUE)[1:10]
```

```
       Rap         R&B        Trap West Coast      France         Pop French Rap
      4997         441         346         247         227         203         155
      Rock     Atlanta East Coast
       152         125         124
```

```
## ------------------------------------------------------------------

# Split up stats data frame
nodes_study$annotations    <- nodes_study$stats$Annotations
nodes_study$suggestions    <- nodes_study$stats$Suggestions
nodes_study$transcriptions <- nodes_study$stats$Transcriptions
nodes_study$questions      <- nodes_study$stats$Questions
nodes_study$answers        <- nodes_study$stats$Answers
nodes_study$pyongs         <- nodes_study$stats$Pyongs
nodes_study <- subset(nodes_study, select = -stats)

# Roles to boolean variables
all_roles <- sort(unique(unlist(nodes_study$roles)))
for (r in all_roles) {
  col <- paste0("role_", make.names(r))
  nodes_study[[col]] <- vapply(nodes_study$roles, function(x) r %in% x, logical(1))
}
nodes_study <- subset(nodes_study, select = -roles)

# Full graph for modeling: isolates KEPT
g <- graph_from_data_frame(d = edges_study, vertices = nodes_study, directed = TRUE)

c(nodes = vcount(g), edges = ecount(g),
  isolates = sum(degree(g) == 0), loops = sum(which_loop(g)))
```

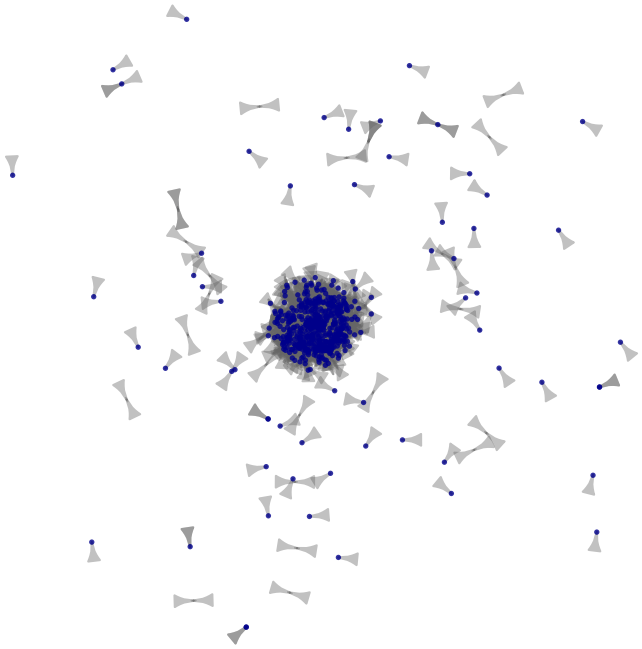
```
   nodes    edges isolates    loops
    9121    17231     5802        0
```

```
## ---- CHANGED: plot only the connected part, report the isolates ----
n_iso  <- sum(degree(g) == 0)
g_plot <- delete_vertices(g, which(degree(g) == 0))   # g itself is untouched

set.seed(585)
lay <- layout_with_fr(g_plot)

par(mar = c(0, 0, 0, 0))
plot(g_plot,
```

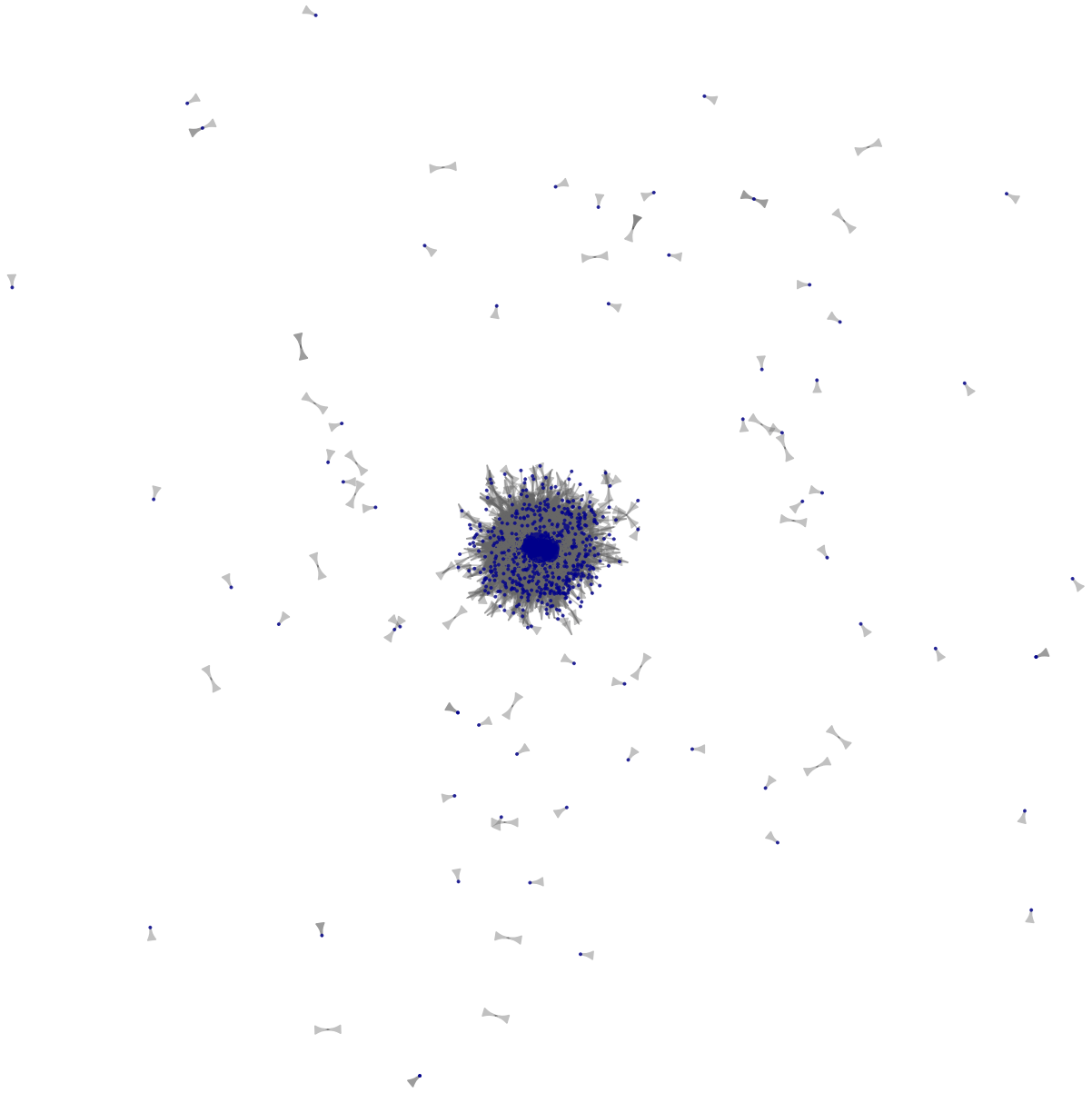
```
     layout = lay,
     vertex.label = NA,
     vertex.size = degree(g_plot, mode = "in")/60,
     vertex.color = adjustcolor("darkblue", 0.8),
     vertex.frame.color = NA,
     edge.arrow.size = 0.5,
     edge.color = adjustcolor("grey40", 0.4))
title(sub = paste(n_iso, "isolated nodes omitted from this figure"))
```



Enlarge Plot

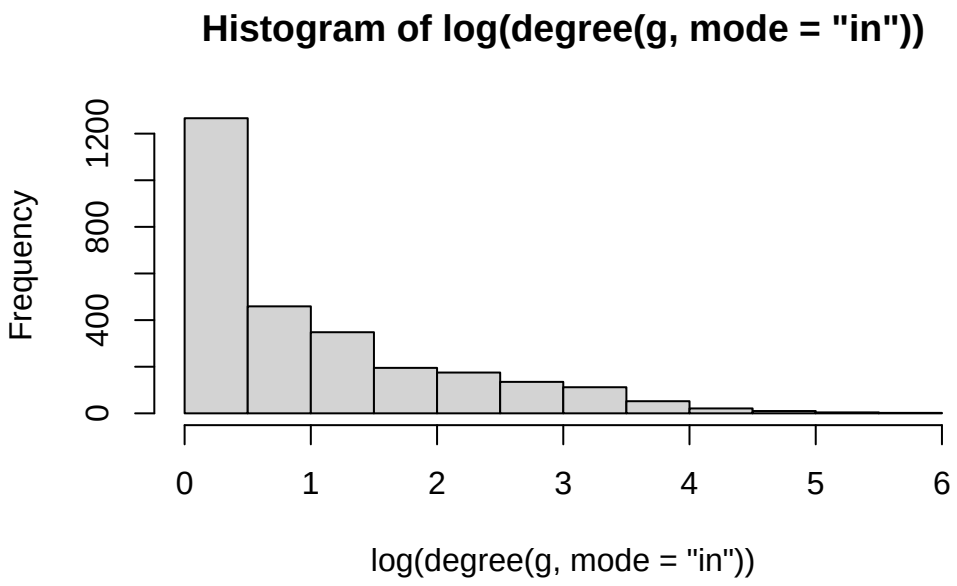
```
par(mar = c(0, 0, 0, 0))

plot(g_plot,
     layout = lay,
     vertex.label = NA,
     vertex.size = degree(g_plot, mode = "in")/60,#2 + 2 * sqrt(degree(g_plot, mode = "in"))
     vertex.color = adjustcolor("darkblue", 0.8),
     vertex.frame.color = NA,
     edge.arrow.size = 0.5,
     edge.color = adjustcolor("grey40", 0.4))
title(sub = paste(n_iso, "isolated nodes omitted from this figure"))
```



Descriptive Analysis

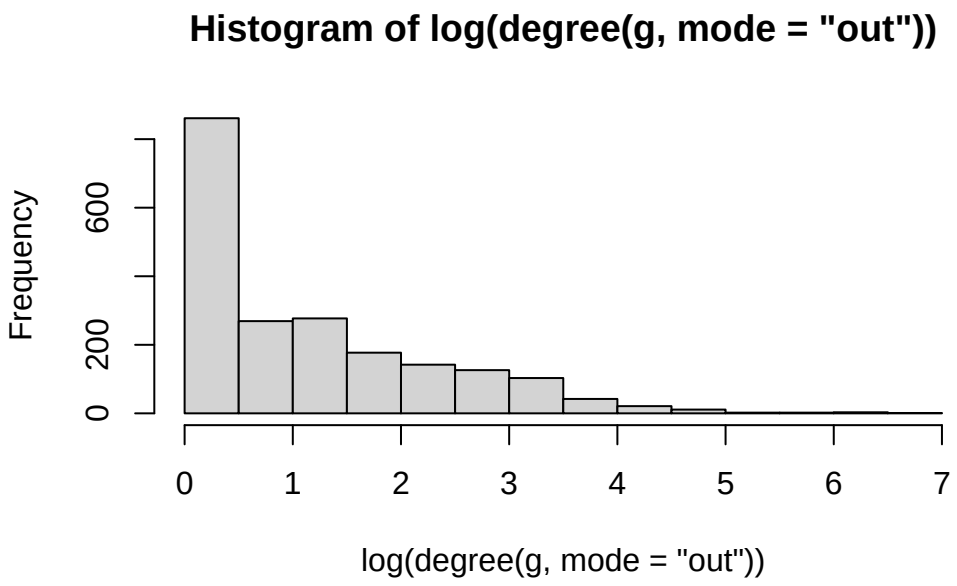
```
# A. Node Degree Distribution
hist(log(degree(g, mode = "in")))
```




```
table(degree(g, mode = "in"))
```

```
   0    1    2    3    4    5    6    7    8    9   10   11   12   13   14   15
6342 1266  459  212  136   77   60   58   49   34   41   27   24   24   23   20
  16   17   18   19   20   21   22   23   24   25   26   27   28   29   30   31
  10   17   16   16    9   15   14   13    9    5   15    5   13    7    5    5
  32   33   34   35   36   37   38   39   40   41   42   43   45   46   47   48
   3    3    2    2    2    5    4    6    4    6    1    1    2    6    1    3
  49   51   52   53   55   56   57   59   60   62   64   68   70   71   74   83
   1    3    1    2    2    3    1    1    2    3    1    2    1    1    1    1
  84   89   92   95   97  103  109  116  127  135  140  158  195  199  208  267
   1    1    1    1    1    1    1    1    2    1    1    1    1    1    1    1
 285
   1
```

```
hist(log(degree(g, mode = "out")))
```




```
table(degree(g, mode = "out"))
```

```
   0    1    2    3    4    5    6    7    8    9   10   11   12   13   14   15
7084  861  269  172  105   73   52   52   37   38   24   26   17   16   22   20
  16   17   18   19   20   21   22   23   24   25   26   27   28   29   30   31
  14   15   12   13   14   15    6    7    6   12    5    9    6    9    8   10
  32   33   34   35   36   37   38   40   41   42   43   44   45   46   47   48
   3    7    1    6    2    2    3    4    2    3    2    4    3    3    1    1
  49   50   51   55   57   59   61   65   66   67   68   69   70   71   75   76
   2    1    2    1    1    1    2    2    2    2    1    1    1    1    1    1
  79   82   84   91   94   98  104  107  109  111  130  132  142  146  163  243
   2    1    1    1    1    1    1    1    1    1    1    1    1    1    1    1
 249  284  407  421  602  891
   1    1    1    1    1    1
```

```
mean(degree(g, mode = "in"))
```

```
[1] 1.889157
```

```
sd(degree(g, mode = "in"))
```

```
[1] 8.909209
```

```r
mean(degree(g, mode = "out"))
```

```
[1] 1.889157
```

```r
sd(degree(g, mode = "out"))
```

```
[1] 15.35645
```

```r
# B. Global and Local Density
#Global
edge_density(g)
```

```
[1] 0.0002071444
```

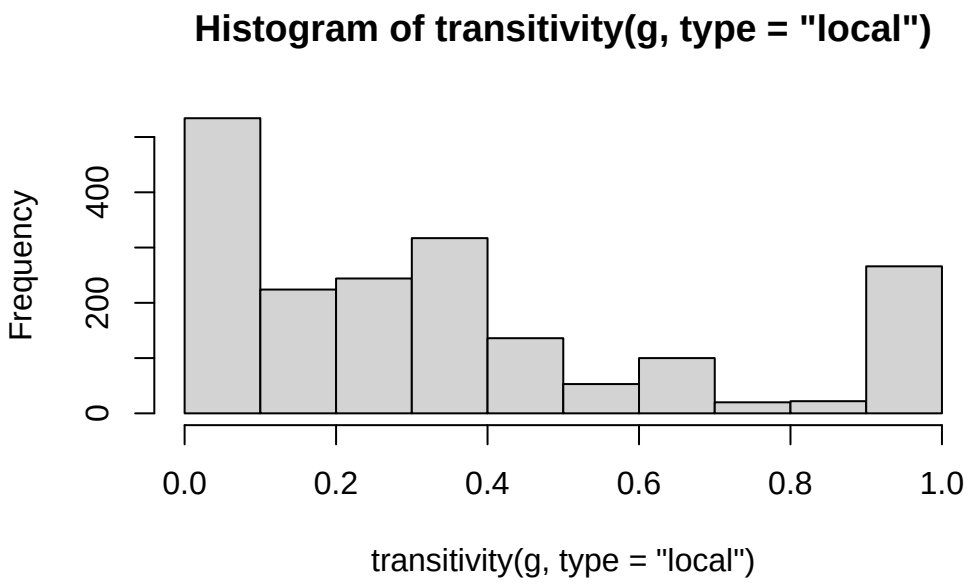
```r
#Local (Ego-Network Density)
ego1 <- induced_subgraph(g, c(1, neighbors(g, 1)))
egohalf <- induced_subgraph(g, c(round(vcount(g)/2), neighbors(g, round(vcount(g)/2))))
edge_density(ego1)
```

```
[1] 0.3333333
```

```r
edge_density(egohalf)#Must be an Isolate
```

```
[1] NaN
```

```r
#C. Clustering Coefficient
hist(transitivity(g, type = "local"))
```




```
transitivity(g, type = "global")
```

```
[1] 0.07632792
```

```
mean(na.omit(transitivity(g, type = "local")))
```

```
[1] 0.3508163
```

```
sd(na.omit(transitivity(g, type = "local")))
```

```
[1] 0.3289256
```

```
# D: Average Betweenness
mean(betweenness(g))
```

```
[1] 1228.311
```

```
#E : Degree/Nominal Assortativity
#Degree Assortativity
assortativity_degree(g, directed = TRUE)
```

```
[1] -0.1983353
```

```r
#Nominal Assortativity
assortativity_nominal(g,types = as.numeric(factor(V(g)$tag)), directed = TRUE)
```

```
[1] 0.06087037
```

```r
#F: Graph Partinitioning
#cluster_fast_greedy(g)
#Only works on undirected graphs
#Havent learned a community detection approach for directed so heres one from Claude:
g_core <- delete_vertices(g, which(degree(g) == 0))   # isolates would each become their own

set.seed(585)
im <- cluster_infomap(g_core, nb.trials = 10)

length(im); sort(sizes(im), decreasing = TRUE)[1:10]
```

```
[1] 400

Community sizes
 10   5   2  41  34   6  18  73   1  35
678 389 258  74  61  55  39  36  33  33
```

```r
modularity(g_core, membership(im), directed = TRUE)   # directed modularity
```

```
[1] 0.33597
```

```r
# does the partition line up with genre?
compare(membership(im), as.integer(factor(V(g_core)$tag)), method = "nmi")
```

```
[1] 0.2767029
```